OnboardAI

Project Handover & Learning Document

Prepared for	Engineering, Test and Quality Assurance teams
Subject	AI-assisted employee onboarding platform
Status	Prototype / internal demonstrator — NOT production ready
Date	06 September 2026
Basis	Direct inspection of the working tree and running system

This document exists so that a team can learn from the project as it actually stands, not as anyone would like it to stand. Every claim below was checked against the code or against a running instance. Where the implementation is weak, incomplete or misleading, that is stated plainly. Where it is genuinely good, that is stated too.

HOW TO READ THIS DOCUMENT

Every topic is covered twice. **Section A** explains the idea in ordinary English, with no jargon, for anyone who needs to understand what the thing does and why it matters. **Section B** covers the same ground in technical terms, with file names, metrics and design trade-offs, suitable for an engineering or academic audience.

1. Executive Summary

SECTION A — IN PLAIN ENGLISH

OnboardAI is a system for managing new employees joining a company. When someone is hired there is a mountain of admin: identity documents to check, accounts to create, training to book, policies to read. Today a human in HR does most of this by hand, and new joiners spend their first fortnight confused and waiting.

This project tries to automate the tedious parts and predict the problems. It reads uploaded documents and decides whether they are valid, answers new joiners' questions from the company's own policy handbook, and flags which new starters look likely to get stuck before anyone in HR notices.

The honest headline: the intelligent parts work and are better built than one might expect. The engineering discipline around them does not exist yet. There are no automated tests at all, and the system has no security whatsoever — anyone who can reach the server can read any employee's passport scan. It is a strong prototype and an unsafe product, and those two facts are both true at the same time.

SECTION B — TECHNICAL DETAIL

Full-stack application: React 19 / Vite / Tailwind frontend, FastAPI / SQLAlchemy / SQLite backend, with five machine-learning and NLP capabilities wired into the request path rather than bolted on as demos.

Capability	Technique	Headline metric	State
Document intelligence	TF-IDF (word+char) → calibrated LinearSVC	0.986 acc / 0.937 on degraded scans	Working
Intent routing	TF-IDF → logistic regression, 8 classes	0.937 acc, CV 0.933 ± 0.004	Working
Sentiment	TF-IDF → logistic regression, 3 classes	0.996 — inflated, see F-07	Working, over-stated
Retrieval (RAG)	Hybrid TF-IDF + LSA, access-controlled	No formal eval — see F-11	Working, unmeasured
Stall-risk	Logistic regression + contribution explain	ROC-AUC 0.794, CV 0.810 ± 0.002	Working
Agent	LangGraph tool loop, 5 tools, memory	No eval harness — see F-12	Working, unmeasured

Five findings are Critical or High severity and at least two of them would block any release review at a competent organisation. They are listed in Section 12.

2. Architecture and How the Pieces Fit

SECTION A — IN PLAIN ENGLISH

Think of the system as three layers. The **front** is the website people see: one view for new joiners, another for HR. The **middle** is a server that holds all the data and enforces the rules. The **back** is a set of small trained models that read documents, understand messages and make predictions.

One design decision is worth understanding because it saved the project. The system never talks to a specific AI company directly. Instead there is a single switch that says which AI service to use — a free one running on the developer's own laptop, or Google's, or another. When Google's free allowance ran out mid-testing, the whole system carried on working by flipping that one switch. Nothing else had to change.

A second decision matters for everything else: every action the system takes is written into a single logbook. Not a human-readable diary — a structured record a computer can count and learn from. The prediction models read that logbook. This is why the parts of the system genuinely feed each other rather than sitting side by side.

SECTION B — TECHNICAL DETAIL

Provider abstraction. `backend/llm_provider.py` is the only module that names a model vendor. `get_llm()` returns a LangChain chat model chosen by the `LLM_PROVIDER` environment variable (ollama / gemini / groq), with imports deferred so an uninstalled fallback never breaks the default path. This was not speculative: when Gemini returned `429 RESOURCE_EXHAUSTED` under test load, switching to local `qwen2.5:7b` required one environment variable and zero code changes. That is the abstraction paying for itself.

Event stream. The `events` table holds structured records (`event_type, actor, payload, confidence, created_at`) and is deliberately distinct from `audit_logs`, which holds human prose for the HR timeline. The risk model's feature builder reads only from `events`, so any action that skips it is invisible to the ML layer. This is a sound separation and the single best structural decision in the codebase.

Request path for a chat message:

```
client → POST /api/chat → intent + sentiment classifiers → PII redaction → LangGraph agent (5 tools, one of which is RAG) → event written → response with agent label, intent, confidence, redaction list
```

Degradation. If no model is reachable the dashboard still functions and the chat states plainly that AI features are offline. `GET /api/health` actively verifies provider reachability rather than merely checking an environment variable is set.

3. Capability One: Reading and Checking Documents

SECTION A — IN PLAIN ENGLISH

When a new joiner uploads a passport or a safety certificate, someone has to check it. Is it actually the document that was asked for, or did they upload the wrong file? Is it in date? Does the name match? Has it been signed?

The system now does that check itself. It reads the text out of the file, works out what kind of document it is, pulls out the dates and names, and applies rules. If everything passes and it is confident, it approves. If something is definitely wrong — an expired certificate, a missing signature, the wrong document entirely — it rejects and says why. Anything in between goes to a human.

That last part is the important bit. The system is deliberately cautious. Approving a bad document by mistake is far more expensive than asking a person to take a look, so anything uncertain is handed over rather than guessed at.

Worth knowing: the model was trained on documents the team generated themselves, because real passports and certificates cannot be collected for training. The accuracy figures therefore describe how well it handles invented documents, not real ones. Nobody has yet tested it on a single genuine document.

SECTION B — TECHNICAL DETAIL

Pipeline in `backend/ml/pipeline.py`: extract → classify → extract fields → validate → route.

Text extraction prefers a PDF's embedded text layer and falls back to OCR only for scans. OCR is an optional dependency, so the pipeline runs without a 2.5 GB PyTorch install and reports honestly when it cannot read an image.

Classifier: TF-IDF over combined word (1-2) and character (3-5) n-grams into a calibrated LinearSVC. The character n-grams are the deliberate choice — they degrade gracefully when OCR damage destroys whole words, which word features cannot survive.

Scan quality tier	What it simulates	Accuracy
clean	Digital PDF, full text layer	1.000
degraded	Header stripped, 3-9% char noise, cross-class bleed	1.000
severe	Cropped phone photo, 10-20% noise, header gone	**0.937**

Overall 0.986 accuracy, macro-F1 0.986, 5-fold CV 0.986 ± 0.004 across 9 classes and 4,950 documents.

Methodological note that reflects well on the work. The first trained model scored 1.000 on every metric with zero cross-validation variance. That was correctly treated as a failure rather than a success: it meant the templates were trivially separable and the metric was meaningless. The corpus was rebuilt with three degradation tiers, cross-class text bleed and an `other` class for junk uploads. The 0.937 severe-tier figure is the honest, defensible number and the one that should be quoted.

Routing policy (`AUTO_APPROVE_CONFIDENCE = 0.80`): a blocking check failure auto-rejects; confidence below 0.60 or any warning routes to human review; only a confident classification with entirely clean checks auto-approves.

Field extraction assigns each date the role of the *nearest preceding label*. This was a real bug found in testing: passports print issue and expiry on one line, and a whole-line keyword search tagged the issue date as the expiry, declaring a valid passport expired.

4. Capability Two: Understanding What People Write

SECTION A — IN PLAIN ENGLISH

New joiners type questions into a chat box. The system reads each message twice: once to work out *what they want*, and once to work out *how they feel*.

Knowing what they want lets the system send the question to the right place — an IT question to the IT helper, a policy question to the policy helper. Knowing how they feel matters more than it sounds. A new joiner who is getting frustrated is a new joiner who might quit in their first month, which is enormously expensive. That frustration score is fed into the prediction model described later.

The system also strips out personal details — email addresses, phone numbers, bank account numbers, national ID numbers — before the message is sent to any outside AI service. Those details are replaced with placeholders and are never stored anywhere.

If someone writes something that suggests real distress, it is flagged to a human immediately regardless of what the message was about.

SECTION B — TECHNICAL DETAIL

Two TF-IDF (word + char n-gram) logistic-regression classifiers in `backend/ml/nlp_models.py`.

Model	Classes	Accuracy	5-fold CV macro-F1	Assessment
Intent	8	0.937	0.933 ± 0.004	Credible
Sentiment	3	0.996	0.996 ± 0.003	**Inflated, see F-07**

Intent confidence floor of 0.45. Below it nothing is routed to a specialist. This exists because a bare "any update?" genuinely does not encode an intent, and the training corpus deliberately contains such messages labelled with an intent the text cannot reveal. That irreducible error is most of the 6.3% gap, which is the correct place for error to live.

Sentiment feeds ML. The classifier emits `p(positive) − p(negative)`, a continuous score written to the event stream and consumed as the `avg_sentiment` feature of the risk model. This is the concrete NLP→ML coupling, and its ablation is measured in Section 7.

PII redaction (`backend/ml/pii.py`) covers email, Aadhaar, PAN, IFSC, passport, phone, account and card patterns, ordered most-specific-first so a generic long-number rule cannot partially consume an IFSC code. Values are replaced with typed placeholders and never returned, so they cannot leak into logs or the event payload. This is rules-only; Presidio's ML-based NER would be the production upgrade.

5. Capability Three: Answering From the Company Handbook

SECTION A — IN PLAIN ENGLISH

When someone asks "how much holiday do I get?", the system must not guess. Holiday entitlement differs between companies, and a confident wrong answer about pay or leave causes real harm.

So the system searches the company's own policy documents, finds the relevant passage, and answers only from what it found — quoting which document and section it used. If it finds nothing relevant, it says so and offers to pass the question to HR. It is explicitly forbidden from filling the gap with general knowledge.

Two employees at two different companies asking the identical question get different, correct answers. An employee in one department cannot retrieve another department's confidential policies.

The clever part. Every question the system could not answer is recorded. HR sees a list: "five people asked about parental leave and nothing in your handbook covers it." A failure of the chatbot becomes a to-do list for improving the company's documentation. The more the system is used, the better the handbook gets.

SECTION B — TECHNICAL DETAIL

`backend/ml/rag.py`. Corpus of 8 documents / 30 chunks, split on markdown sections with the document title prepended so a chunk retrieves on its subject even when the section text never repeats it.

Hybrid scoring: `0.5 × TF-IDF cosine + 0.5 × LSA cosine`. TF-IDF catches exact policy wording ("lockout-tagout"); a TruncatedSVD projection of the same matrix catches paraphrase. Measured example — "can I take time off before my probation ends" against the *Leave during probation* section scores **0.339 lexically but 0.846 semantically**. The semantic half is doing the work, which justifies the hybrid.

LSA rather than neural embeddings so the system runs fully offline with no model download. Given that the environment could not reliably download a 4.7 GB model from a CDN during development, this was the correct engineering call, not a compromise.

Access control is enforced at retrieval: a chunk is visible only if its `org` matches and, where the document restricts one, the employee's department is permitted. Verified: Nimbus and Veltrix employees asking the identical leave question receive 24 and 18 days respectively; a Sales employee retrieves zero chunks from the Engineering information-security document.

Gap detection uses two thresholds. Below `RELEVANCE_FLOOR = 0.12` nothing is returned. Between that and `WEAK_MATCH_SCORE = 0.42` passages are returned but flagged to the model as possibly irrelevant, and the question is logged as a `knowledge.gap` event. The second threshold was added after testing showed the common failure is *topically adjacent*, not empty: asking about parental leave retrieves the leave policy without being covered by it.

6. Capability Four: The Assistant That Can Act

SECTION A — IN PLAIN ENGLISH

The chat assistant is not just a question-answering box. It can do things: look up a status, tick off a task, search the handbook. It remembers the conversation, so "and which of those was verified most recently?" works without repeating context.

There is a security property here worth understanding. The assistant physically cannot look up another employee's record. Not "it has been told not to" — the tools it can use are built around one specific employee when the conversation starts, so there is no way to ask for someone else's data. This was tested by directly instructing it to ignore its rules and fetch a colleague's file; it could not.

Where this is over-sold. The interface displays labels like "IT Provisioning Agent" and "Policy Agent", which implies a team of specialist assistants passing work between them. That is not what happens. One single assistant handles everything; the label shown is chosen by the message classifier. It is honest routing information presented in a way that suggests more architecture than exists.

SECTION B — TECHNICAL DETAIL

`backend/agents/chatbot.py`, built on LangChain 1.x `create_agent` (a LangGraph tool-calling loop) with an `InMemorySaver` checkpointer keyed by employee id for multi-turn memory.

Tool scoping as an authorisation boundary. `build_tools(emp_id, org_id, department)` constructs tools inside a closure over the authenticated identity. `emp_id` is never a model-supplied argument, so no prompt can cause the model to address another record. This is the right pattern and is materially stronger than instructing a model to behave. **It is also undermined by F-01**: the REST API beneath it has no authorisation at all, so the property holds only for the chat path.

Five tools: `get_my_status`, `list_my_tasks`, `list_my_documents`, `complete_task`, `search_company_policy`.

Observed grounding behaviour. Asked when each document was verified — data the tools do not expose — the agent replied that it did not have timestamps and offered to check with HR rather than inventing them. Refusal under pressure to answer is the single most important behaviour for an HR assistant and it was demonstrated, not assumed.

Content-block handling. Gemini 3.x returns message content as a list of typed blocks including internal reasoning. Naive stringification would have leaked the model's reasoning to employees; `_as_text()` extracts only `type == "text"` blocks.

7. Capability Five: Predicting Who Will Get Stuck

SECTION A — IN PLAIN ENGLISH

Some new joiners sail through onboarding. Others quietly stall — they stop responding, their documents keep getting rejected, they go quiet in chat — and by the time HR notices, they are three weeks behind or have already decided to leave. Replacing a new hire costs a great deal.

The system scores every new joiner on how likely they are to stall, and — importantly — explains why. Not just "48% risk" but "48%, driven by no tasks completed, no knowledge-transfer progress, and one rejected document." HR can act on the second version; they cannot act on the first.

Two honest caveats. First, no real onboarding outcomes exist to learn from, so the model was trained on simulated employees. It has never seen a real person. Second, two of the fourteen things it looks at are not actually measured — they are filled in with the same fixed number for everybody, every time. The model was taught that those things vary and then given a constant. That is a genuine defect, not a rounding error.

SECTION B — TECHNICAL DETAIL

Logistic regression selected over histogram gradient boosting on held-out ROC-AUC. **ROC-AUC 0.794**, 5-fold CV **0.810 ± 0.002**, base stall rate 43.3%.

Explanations are exact, not sampled. For a standardised linear model, `coef × scaled_value` *is* the SHAP value, so per-employee attribution is deterministic and needs no KernelExplainer. `shap` is installed and used only if a tree model is ever selected.

Ablations — retraining with one feature removed:

Removed feature	ROC-AUC	Δ	Reading
avg_sentiment	0.7871	–0.0070	Largest single loss; produced by the NLP layer
days_since_last_activity	0.7889	–0.0052	Real, modest signal
docs_rejected	0.7935	–0.0006	Negligible in aggregate — see note

The `avg_sentiment` ablation is the measured evidence that the NLP and ML layers are load-bearing rather than decorative. It is the strongest single argument in the project's favour and should be led with.

The `docs_rejected` result is instructive rather than disappointing: rejections are rare, so removing the feature barely moves an aggregate metric, yet it contributes `+0.242` to the log-odds for the one employee who has one. Aggregate discrimination and per-case explanation answer different questions, and conflating them is a common analytical error.

Generative design. `risk_synth.py` models three latent traits (engagement, friction, support) with observables as noisy measurements and an explicitly unobserved component in the outcome. No feature set can therefore reach 1.0, which is deliberate: a near-perfect score would indicate the generator had leaked the label.

8. The Learning Loops

SECTION A — IN PLAIN ENGLISH

The most commercially interesting idea in the project is that the system is designed to get better the more it is used, without anyone retraining it by hand.

When HR disagrees with the document checker — approving something it rejected, or vice versa — that disagreement is stored as a lesson. The system keeps a running score of how often the humans agreed with it. Those disagreements are exactly the examples needed to train a better version.

Similarly, every question the handbook could not answer becomes a suggestion for what to write next. Answer it once, and the system can answer it forever.

Be clear about what exists: the data is being collected correctly and completely. The step that actually feeds it back into a retrained model has not been built. The loops are wired at one end only.

SECTION B — TECHNICAL DETAIL

Loop 1 — document labels. `document_uploads.verdict` (model) is stored separately from `hr_decision` (human). `GET /api/ml/agreement` reports the live agreement rate. HR can override an auto-approval specifically so this signal keeps being collected. The column separation is a genuinely thoughtful piece of schema design.

Loop 2 — documentation gaps. `knowledge.gap` events aggregate at `GET /api/knowledge/gaps`, ranked by distinct employees then frequency. The corpus is deliberately incomplete (parental leave, notice period, relocation) so the mechanism is demonstrable.

Loop 3 — risk outcomes. Events are captured but no completed-onboarding label is ever written, so this loop is currently notional.

Missing across all three: no retraining trigger, no scheduled job, no model registry, no champion/challenger comparison, no drift detection, and no guard against the feedback loop that arises when a model's own auto-approvals become part of its next training set. That last one is a real methodological trap and nothing currently prevents it.

9. The Developer's Work: An Honest Assessment

SECTION A — IN PLAIN ENGLISH

What was done well. The developer repeatedly refused to accept flattering results. Three separate times a model scored a perfect 100%, and each time this was treated as a bug in the test rather than a triumph — the data was rebuilt to be harder and more realistic. Most people, under deadline, would have screenshotted the 100% and moved on. That instinct is the single most valuable thing on display here.

Several real bugs were caught by writing checks before believing the code: a passport being declared expired because two dates shared a line; a text-matching rule that grabbed part of the word "CERTIFICATE" and treated it as a certificate number; an error message that blamed the network for every failure and so hid genuine bugs behind a plausible excuse.

What was not done. None of those checks were kept. They were written, run, and thrown away. There is not a single automated test in the project. Every one of those bugs could return tomorrow and nobody would know until a demo failed.

There is also no security of any kind. Not weak security — none. Anyone who can reach the server can read any employee's file and download their uploaded passport without so much as a username.

SECTION B — TECHNICAL DETAIL

Strengths, specifically:

- Provider abstraction that demonstrably absorbed a vendor quota failure with a one-line change.
- Closure-scoped agent tools as a real authorisation boundary rather than a prompt instruction.
- Magic-byte file-type verification rather than trusting the client's declared Content-Type; UUID storage paths; path-traversal guard on retrieval.
- verdict / hr_decision column separation, anticipating supervised retraining.
- events vs audit_logs separation, giving ML a machine-readable substrate.
- Evaluation methodology: difficulty-tiered corpora, ablation studies, per-tier reporting, and explicit rejection of ceiling-effect metrics.
- Graceful degradation with an active (not merely configured) health check.

Weaknesses, specifically:

- Zero tests (F-02). Verification was performed via disposable scripts in a temporary directory that form no part of the repository.
- No authentication or authorisation anywhere (F-01).
- Train/serve skew: two of fourteen risk features are hardcoded constants at inference (F-04).
- Presentation overstates architecture — specialist agent labels without specialist agents (F-06).
- No database migrations; schema evolution relies on `create_all` and manual reseeding, which silently loses data.
- Residual mocked flows still shipped in the UI: HR 'Provide Documents' and 'KT' uploads use `prompt()` and persist only to component state; standard-document CRUD is client-side only; 'Download PDF' buttons do nothing.
- Dead code: `mockData.js` exports employees and organizations that nothing imports; a second empty virtualenv at the repository root.
- Pydantic v1 `orm_mode` key still present in `schemas.py`, emitting a deprecation warning on every boot.

- Frontend ships as a single 669 KB bundle with no code splitting.

10. The Tester's View

SECTION A — IN PLAIN ENGLISH

The blunt position: this project has never been tested in any repeatable sense. It has been demonstrated. Those are different things.

During development, checks were written and run by hand, and they did find genuine faults. But they were temporary files in a scratch folder. They are not in the project. Nobody else can run them. They will not run tomorrow. If a change breaks the document checker next week, the first person to find out will be whoever is presenting.

A tester joining this team would start from nothing: no test suite, no test data strategy, no environment to test against, no definition of what "working" means, and no way to tell whether today's build is better or worse than yesterday's.

There is one further problem specific to AI systems. Ordinary software gives the same answer every time, so you can check it against an expected result. This system's assistant phrases things differently on every run. You cannot test it by comparing text. You have to test the properties that must always hold — did it cite a source, did it refuse to invent a policy, did it stay within the right employee's data — and that requires a different kind of test that nobody has written yet.

SECTION B — TECHNICAL DETAIL

Current coverage: 0%. No `pytest`, no `vitest`, no `conftest.py`, no CI configuration, no fixtures. A repository-wide search returns test files only inside `venv/site-packages`.

Minimum viable test suite, in priority order:

Layer	Tool	What must be covered	Priority
Security	pytest + TestClient	Every endpoint rejects unauthenticated access; an employee token cannot read another employee's record or upload	P0
Upload safety	pytest	Magic-byte rejection of disguised files; >10 MB rejection; path traversal in stored_path; empty file	P0
Pipeline contract	pytest	The 7 demo PDFs each produce their expected verdict; expired certificate rejects with a reason; unsigned offer letter rejects	P0
ML regression	pytest + thresholds	Fail the build if doc-classifier macro-F1 < 0.95, severe-tier < 0.90, intent < 0.90, risk ROC-AUC < 0.75	P1
Retrieval	pytest	Org isolation; department restriction; known-answer retrieval; gap logged below threshold	P1
Agent properties	pytest + LLM judge	Cites a source when answering policy; declines when corpus lacks the answer; cannot address another emp_id under injection	P1
API contract	schemathesis	OpenAPI fuzzing for 500s on malformed input	P2
Frontend	vitest + Testing Library	Upload states, risk badge rendering, decision flow	P2
End to end	Playwright	Login → upload → HR review → decision recorded	P2

Non-determinism strategy. Assert invariants, never exact strings. For agent tests, pin `LLM_PROVIDER=ollama` and `temperature=0` for repeatability, and separate deterministic assertions (a tool was called; a citation marker is present; the DB row changed) from qualitative ones (the answer is helpful), which need an LLM-as-judge rubric and should be reported as a trend, not a pass/fail gate.

Missing test data. There is no fixture database, no factory, and tests would currently mutate the live `onboardai.db`. A session-scoped SQLite fixture with transaction rollback per test is the first infrastructure a tester should build.

11. The Quality Assurance View

SECTION A — IN PLAIN ENGLISH

Testing asks "does this work?". Quality assurance asks "is the way we build this capable of producing something trustworthy?". On that second question the answer is currently no, and it is worth separating that from the quality of the thinking, which is high.

There is no process. No definition of when something is finished, no review by a second person, no automated checks before code is accepted, no record of decisions, no version control history to learn from. The work is good; the system that produced it would not reliably produce good work again, and would not catch it if it did not.

For an AI product specifically, there are obligations beyond ordinary software. This system makes decisions about people — rejecting their documents, scoring their likelihood of failing. Those decisions need to be explainable, appealable, monitored for bias, and documented. Some of that exists by happy accident: the risk score explains itself, and humans can override the document checker. None of it exists by design or policy.

Most seriously: a system that stores scans of passports and bank details, and sends employee messages to a third party, has legal obligations. Nobody has assessed those.

SECTION B — TECHNICAL DETAIL

Process gaps:

- No version control in use — the working tree is not a git repository, so there is no history, no blame, no revert, no branch protection and no review gate.
- No CI pipeline; lint exists but is advisory and never blocks.
- No definition of done, no acceptance criteria, no requirements traceability.
- No dependency pinning: `requirements.txt` is unversioned, so builds are not reproducible. A LangChain 1.x major-version change already broke the original agent code silently — it only ever hit its exception branch.
- No structured logging strategy, no error tracking, no metrics, no alerting.

Responsible-AI gaps specific to this product:

- **No fairness evaluation.** The risk model consumes department and seniority. Nobody has tested for disparate impact across those or any correlated attribute. A model that systematically scores one department as high-risk would be indistinguishable from a working one today.
- **No model cards** documenting intended use, training data, limitations or out-of-scope uses.
- **No appeal path** defined for an employee whose document is auto-rejected.
- **No drift monitoring.** Models trained on synthetic data are especially exposed to real-world distribution shift, and nothing would detect it.
- **No human-review SLA.** Documents routed to needs_review can sit indefinitely; nothing tracks or escalates them.
- **No DPIA.** Identity documents and bank details are stored, and message text is transmitted to a third-party model provider by default. PII redaction reduces but does not eliminate this, and the legal basis has not been assessed.
- **Retention undefined.** Uploaded identity documents are never deleted; there is no retention policy or deletion path.

Quality gates that should exist before any pilot: authentication on every endpoint; test coverage floor; ML metric regression thresholds in CI; dependency pinning and vulnerability scan; secrets never in client bundles; DPIA sign-off; documented rollback.

12. Findings Register

Ordered by severity. Every item was verified against the code or a running instance during preparation of this document.

F-01 — CRITICAL **No authentication or authorisation on any API endpoint**

Every route in `routes/api.py`, `routes/documents.py` and `routes/chat.py` declares `Depends(get_db)` and nothing else. Anyone able to reach port 8000 can enumerate all employees, read any risk score, change any onboarding stage, approve or reject any document, and download any uploaded file — including passport and bank-detail scans — via `GET /api/uploads/{id}/file`. The agent's careful tool scoping is bypassed entirely by calling the REST API directly. **This alone makes the system unfit to hold real data.**

F-02 — CRITICAL **No automated tests exist**

No pytest, vitest, CI configuration or fixtures anywhere in the project. Verification was performed with disposable scripts in a temporary scratch directory which are not part of the repository and cannot be re-run by anyone else. Several real defects were found this way and would not be caught again if reintroduced. There is no regression safety net of any kind.

F-03 — CRITICAL **Credentials hardcoded in client-side JavaScript**

`src/pages/Login.jsx` compares username and password literals in the browser bundle (`employee 1/@1`, `hr1/@hr1`). They are visible to anyone who opens developer tools or reads `dist/`. There is no session, no token, and no server-side verification — 'login' only sets React state.

F-04 — HIGH **Train/serve skew: two risk features are constants at inference**

`risk_model.py` lines 148 and 150 hardcode `response_latency_hours = 6.0` and `is_remote = 0.0` for every employee. The model was trained on varying values for both. Two of fourteen features therefore carry no information at serving time while still consuming their learned coefficients, which shifts every prediction by a constant and silently degrades the score. Either measure them or remove them from the feature set and retrain.

F-05 — HIGH **All models are trained and evaluated exclusively on synthetic data**

Document classifier, intent, sentiment and risk models are trained on generated corpora. No real document, message or onboarding outcome has ever been seen. The reported metrics describe internal consistency with a generator, not real-world capability, and should never be quoted without that qualification. Generator design is unusually careful (latent variables, difficulty tiers, injected ambiguity), which mitigates but does not remove this.

F-06 — HIGH **Multi-agent architecture is presented but not implemented**

The UI displays 'IT Provisioning Agent', 'Policy Agent' and similar labels, implying specialist agents. In reality a single LangGraph agent holds all five tools and answers everything; the label is chosen by the intent classifier and is cosmetic. The routing information is real, but the presentation implies an architecture that does not exist.

F-07 — HIGH **Sentiment accuracy of 0.996 is a measurement artefact**

The figure reflects template separability, not sentiment understanding. It moved from 0.975 to 1.000 to 0.996 purely through changes to the generator. It must not be presented as evidence the model understands employee sentiment, and requires validation against a hand-labelled sample of real messages before any weight is placed on it — particularly since it feeds the risk model.

F-08 — HIGH **No database migrations**

Schema is created by `Base.metadata.create_all`. Column changes require dropping and reseeding, which destroys data. `seed.py` deletes every table including uploads and events. There is no Alembic setup and no upgrade path for an existing deployment.

F-09 — MEDIUM **Unpersisted and mocked flows still present in the UI**

HR 'Provide Documents' and 'Knowledge Transfer' uploads use `prompt()` and store to component state, so they vanish on refresh. Organisational standard documents are client-side state only. 'Download PDF' buttons are inert. These read as working features in a demonstration.

F-10 — MEDIUM **No rate limiting, pagination or request size limits on API endpoints**

`/api/risk/overview` scores every employee synchronously on each call and will degrade linearly with headcount. No endpoint paginates. Nothing prevents abuse of the chat endpoint, which costs money when a paid provider is configured.

F-11 — MEDIUM **Retrieval quality is unmeasured and thresholds are hand-tuned**

`RELEVANCE_FLOOR = 0.12`, `WEAK_MATCH_SCORE = 0.42`, `LEXICAL_WEIGHT = 0.5` and `k = 3` were chosen by inspection on a handful of queries. There is no labelled query set and no hit-rate, MRR, nDCG or faithfulness measurement.

F-12 — MEDIUM **No agent evaluation harness**

Tool-call correctness, citation compliance, refusal behaviour and injection resistance were checked manually once. There is no repeatable measurement, so regressions in agent behaviour after a prompt or model change would be invisible.

F-13 — MEDIUM **Dependencies unpinned and dead code retained**

`requirements.txt` specifies no versions, so builds are not reproducible — a LangChain major version already broke the agent silently. `mockData.js` retains ~170 lines of unused fixtures; a second empty virtualenv sits at the repository root; `schemas.py` still uses Pydantic v1 `orm_mode`.

13. What Is Genuinely Good

A findings register read alone gives a false impression. These are real strengths and several are above the level typically seen in a prototype.

Strength	Why it matters
Rejecting flattering metrics	Three models initially scored 1.000 and each was treated as a broken benchmark rather than a result. The corpora were rebuilt harder. This is the single most transferable habit in the project.
Provider abstraction	Absorbed a live vendor quota failure with a one-line environment change. Designed before it was needed and immediately vindicated.
Closure-scoped agent tools	Authorisation enforced by construction rather than by instructing a model. Survived a direct prompt injection attempt in testing.
verdict / hr_decision separation	Model prediction and human ground truth kept apart, making supervised improvement possible later. Most prototypes overwrite one with the other.
Ablation study	Provides measured evidence that the NLP layer contributes to the ML layer, rather than asserting integration.
Difficulty-tiered evaluation	Reporting 0.937 on severe scans alongside 1.000 on clean ones is far more informative and more honest than a single headline number.
Defensive file handling	Magic-byte verification, UUID paths, size caps and traversal guards — correct on the first attempt, which is unusual.
Graceful degradation	The product remains usable with no model available, and the health check actively probes rather than assuming.
Honest error reporting	An error handler that blamed the network for every failure was identified as hiding real bugs and corrected to distinguish rate limits, connectivity and genuine faults.

14. What To Do Next

SECTION A — IN PLAIN ENGLISH

If this were a real product with a real deadline, the order of work would be: make it safe, make it provable, then make it cleverer. It is tempting to add more intelligence because that is the interesting part, but adding features to a system with no security and no tests makes the eventual clean-up larger, not smaller.

SECTION B — TECHNICAL DETAIL

#	Action	Addresses	Effort
1	Put the project under version control before anything else	QA process	Minutes
2	Add authentication and per-request authorisation on every endpoint; protect the upload download route	F-01, F-03	2-3 days
3	Build the P0 test suite: security, upload safety, pipeline contract, with a SQLite fixture and rollback	F-02	3-4 days
4	Pin dependencies; add CI running lint, tests and ML metric thresholds	F-02, F-13	1 day
5	Fix train/serve skew: measure the two features or drop them and retrain	F-04	0.5 day
6	Introduce Alembic and stop destructive reseeding	F-08	1 day
7	Either implement real specialist agents or relabel the UI honestly	F-06	1-3 days
8	Build a labelled retrieval query set; measure hit-rate, MRR and faithfulness	F-11	2 days
9	Build the agent eval harness (tool accuracy, citation compliance, injection)	F-12	2 days
10	Persist the remaining mocked flows or remove them from the UI	F-09	1 day
11	Hand-label a few hundred real messages; revalidate sentiment	F-05, F-07	2 days
12	Write model cards; define appeal path, retention policy and review SLA; run a DPIA	QA / Responsible AI	3-5 days

Closing assessment. The intelligent core of this system is thoughtfully built and honestly evaluated, and the evaluation methodology in particular is stronger than the norm for work at this stage. The surrounding engineering — security, testing, reproducibility, process — is absent rather than weak. Both statements are true, and a reader who takes away only one of them has misunderstood the project.